

L24: Pointer Analysis

Software Analysis

IIT Guwahati · September 29, 2026

Running Example: An Elevator (1/3)

```
class Elevator {  
    Object[] floors;  
    Object[] events;  
}
```

A **Java** program.

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
  
    v.floors = new Object[M];  
    v.events = new Object[N];  
  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

Running Example: An Elevator (2/3)

```
class Elevator {  
    Object[] floors;  
    Object[] events;  
}
```

A **Java** program.

An Elevator owns two arrays:
one Floor per storey, one
Event per button press.

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
  
    v.floors = new Object[M];  
    v.events = new Object[N];  
  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

Running Example: An Elevator (3/3)

```
class Elevator {  
    Object[] floors;  
    Object[] events;  
}
```

A **Java** program.

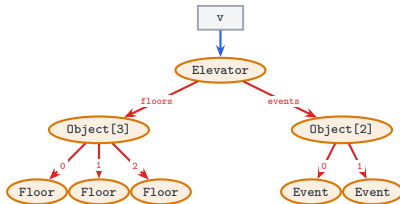
An Elevator owns two arrays:
one Floor per storey, one
Event per button press.

doit is called with *any* M and N.

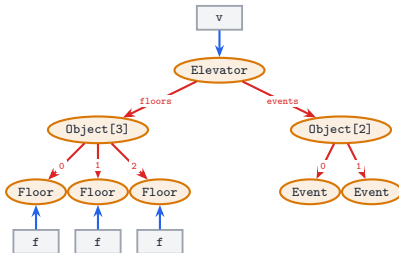
The analysis must cover all of
them at once.

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
  
    v.floors = new Object[M];  
    v.events = new Object[N];  
  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

One Concrete Run: $M = 3$, $N = 2$ (1/3)

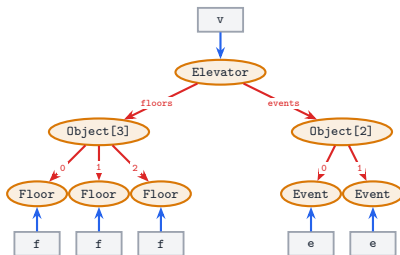


One Concrete Run: $M = 3$, $N = 2$ (2/3)



Six objects, and the array cells are distinguished by index.

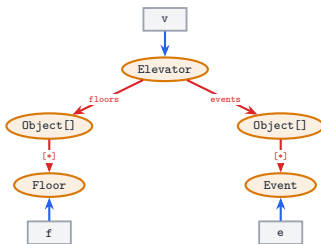
One Concrete Run: $M = 3, N = 2$ (3/3)



Six objects, and the array cells are distinguished by index. With $M = 1000$ the picture has a thousand `Floor` nodes.

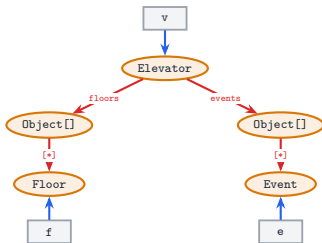
Abstracting the Heap: Allocation Sites (1/3)

One abstract object per **allocation site** — one per new expression.



Abstracting the Heap: Allocation Sites (2/3)

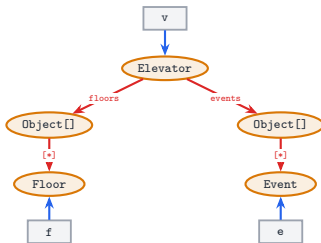
One abstract object per **allocation site** — one per new expression.



Five sites, so **five ovals** — whatever M and N are.

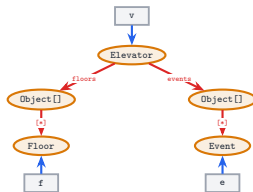
Abstracting the Heap: Allocation Sites (3/3)

One abstract object per **allocation site** — one per new expression.



Five sites, so **five ovals** — whatever `M` and `N` are. Indices become `[*]`: any cell.

The Points-to Graph (1/6)

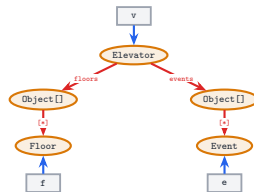


Naik, CIS 5470, Penn — Pointer Analysis

The Points-to Graph (2/6)



a **variable**



Naik, CIS 5470, Penn — Pointer Analysis

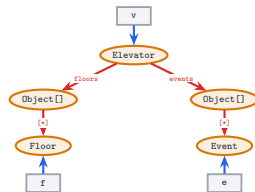
The Points-to Graph (3/6)



a **variable**



an **allocation site**



The Points-to Graph (4/6)



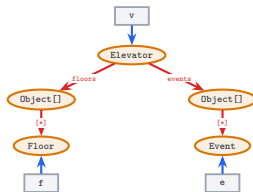
a **variable**



an **allocation site**



v points to the object



The Points-to Graph (5/6)



a **variable**



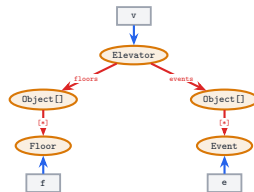
an **allocation site**



v points to the object



its field f points to the object



The Points-to Graph (6/6)



a **variable**



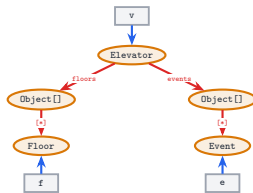
an **allocation site**



v points to the object



its field f points to the object



Directed, because pointing is one-way. Finite, because the ovals are.

Abstracting Control Flow: Flow Insensitivity (1/5)

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
    v.floors = new Object[M];  
    v.events = new Object[N];  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

Abstracting Control Flow: Flow Insensitivity (2/5)

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
    v.floors = new Object[M];  
    v.events = new Object[N];  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

```
v = new Elevator  
v.floors = new Object[]  
v.events = new Object[]  
f = new Floor  
v.floors[*] = f  
e = new Event  
v.events[*] = e
```

Abstracting Control Flow: Flow Insensitivity (3/5)

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
    v.floors = new Object[M];  
    v.events = new Object[N];  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

```
v = new Elevator  
v.floors = new Object[]  
v.events = new Object[]  
f = new Floor  
v.floors[*] = f  
e = new Event  
v.events[*] = e
```

1. No loops, no order — an unordered *set*.

Abstracting Control Flow: Flow Insensitivity (4/5)

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
    v.floors = new Object[M];  
    v.events = new Object[N];  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

```
v = new Elevator  
v.floors = new Object[]  
v.events = new Object[]  
f = new Floor  
v.floors[*] = f  
e = new Event  
v.events[*] = e
```

1. No loops, no order — an unordered *set*.
2. Statements that touch no pointer are gone.

Abstracting Control Flow: Flow Insensitivity (5/5)

```
void doit(int M, int N) {  
    Elevator v = new Elevator();  
    v.floors = new Object[M];  
    v.events = new Object[N];  
    for (int i = 0; i < M; i++) {  
        Floor f = new Floor();  
        v.floors[i] = f;  
    }  
    for (int i = 0; i < N; i++) {  
        Event e = new Event();  
        v.events[i] = e;  
    }  
}
```

```
v = new Elevator  
v.floors = new Object[]  
v.events = new Object[]  
f = new Floor  
v.floors[*] = f  
e = new Event  
v.events[*] = e
```

1. No loops, no order — an unordered *set*.
2. Statements that touch no pointer are gone.
3. Indices become `[*]`.

A Normal Form for Pointer Statements (1/6)

```
s ::= v = new T
    | v1 = v2
    | v1 = v2.f
    | v1.f = v2
    | v1 = v2[*]
    | v1[*] = v2
```

A Normal Form for Pointer Statements (2/6)

<code>s ::= v = new T</code>	object allocation
<code>v1 = v2</code>	
<code>v1 = v2.f</code>	
<code>v1.f = v2</code>	
<code>v1 = v2[*]</code>	
<code>v1[*] = v2</code>	

A Normal Form for Pointer Statements (3/6)

<code>s ::= v = new T</code>	object allocation
<code> v1 = v2</code>	object copy
<code> v1 = v2.f</code>	
<code> v1.f = v2</code>	
<code> v1 = v2[*]</code>	
<code> v1[*] = v2</code>	

A Normal Form for Pointer Statements (4/6)

<code>s ::= v = new T</code>	object allocation
<code> v1 = v2</code>	object copy
<code> v1 = v2.f</code>	field read
<code> v1.f = v2</code>	field write
<code> v1 = v2[*]</code>	
<code> v1[*] = v2</code>	

A Normal Form for Pointer Statements (5/6)

<code>s ::= v = new T</code>	object allocation
<code> v1 = v2</code>	object copy
<code> v1 = v2.f</code>	field read
<code> v1.f = v2</code>	field write
<code> v1 = v2[*]</code>	array read
<code> v1[*] = v2</code>	array write

A Normal Form for Pointer Statements (6/6)

<code>s ::= v = new T</code>	object allocation
<code>v1 = v2</code>	object copy
<code>v1 = v2.f</code>	field read
<code>v1.f = v2</code>	field write
<code>v1 = v2[*]</code>	array read
<code>v1[*] = v2</code>	array write

`v` is a reference-typed variable, `f` a reference-typed field. **Six forms, and everything else decomposes into them.**

Decomposing Compound Statements (1/4)

A source line can touch the heap more than once. Normal form allows **one** access per line, so the address **in between** needs a name — the temporary.

As written

```
v.events = new Object[N];
```

```
v.events[*] = e;
```

Normal form

Decomposing Compound Statements (2/4)

A source line can touch the heap more than once. Normal form allows **one** access per line, so the address **in between** needs a name — the temporary.

As written

```
v.events = new Object[N];
```

```
v.events[*] = e;
```

Normal form

```
tmp2 = new Object[]
```

```
v.events = tmp2
```

Decomposing Compound Statements (3/4)

A source line can touch the heap more than once. Normal form allows **one** access per line, so the address **in between** needs a name — the temporary.

As written

```
v.events = new Object[N];
```

```
v.events[*] = e;
```

Normal form

```
tmp2 = new Object[]
```

```
v.events = tmp2
```

```
tmp4 = v.events
```

```
tmp4[*] = e
```

Decomposing Compound Statements (4/4)

A source line can touch the heap more than once. Normal form allows **one** access per line, so the address **in between** needs a name — the temporary.

As written

```
v.events = new Object[N];
```

```
v.events[*] = e;
```

Normal form

```
tmp2 = new Object[]
```

```
v.events = tmp2
```

```
tmp4 = v.events
```

```
tmp4[*] = e
```

The Elevator in Normal Form (1/4)

Flow-insensitive set

```
v = new Elevator
v.floors = new Object[]
v.events = new Object[]
f = new Floor
v.floors[*] = f
e = new Event
v.events[*] = e
```

Normal form

```
v = new Elevator
```

```
f = new Floor
```

```
e = new Event
```

The Elevator in Normal Form (2/4)

Flow-insensitive set

```
v = new Elevator
```

```
v.floors = new Object[]
```

```
v.events = new Object[]
```

```
f = new Floor
```

```
v.floors[*] = f
```

```
e = new Event
```

```
v.events[*] = e
```

Normal form

```
v = new Elevator
```

```
tmp1 = new Object[]
```

```
v.floors = tmp1
```

```
tmp2 = new Object[]
```

```
v.events = tmp2
```

```
f = new Floor
```

```
e = new Event
```

The Elevator in Normal Form (3/4)

Flow-insensitive set

```
v = new Elevator
v.floors = new Object[]
v.events = new Object[]
f = new Floor
v.floors[*] = f
e = new Event
v.events[*] = e
```

Normal form

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```

The Elevator in Normal Form (4/4)

Flow-insensitive set

```
v = new Elevator
v.floors = new Object[]
v.events = new Object[]
f = new Floor
v.floors[*] = f
e = new Event
v.events[*] = e
```

Normal form

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```

Eleven statements, each matching exactly one of the six forms.

Now the algorithm can run.

Quiz: Normal Form (1/4)

Rewrite each Java statement using only the six forms.

```
v1.f = v2.f;
```

```
v1.f.g = v2.h;
```

Quiz: Normal Form (2/4)

Rewrite each Java statement using only the six forms.

```
v1.f = v2.f;
```

```
tmp = v2.f
```

```
v1.f = tmp
```

```
v1.f.g = v2.h;
```

Quiz: Normal Form (3/4)

Rewrite each Java statement using only the six forms.

```
v1.f = v2.f;
```

```
tmp = v2.f
```

```
v1.f = tmp
```

```
v1.f.g = v2.h;
```

```
tmp1 = v1.f
```

```
tmp2 = v2.h
```

```
tmp1.g = tmp2
```

Quiz: Normal Form (4/4)

Rewrite each Java statement using only the six forms.

```
v1.f = v2.f;
```

```
tmp = v2.f
```

```
v1.f = tmp
```

```
v1.f.g = v2.h;
```

```
tmp1 = v1.f
```

```
tmp2 = v2.h
```

```
tmp1.g = tmp2
```

Count the dereferences: the second statement has **three**, so it needs **three** statements.

Andersen's Algorithm (1/3)

```
graph = empty
repeat:
    for (each statement s in program):
        apply the rule for s to graph
until graph stops changing
```

Andersen's Algorithm (2/3)

```
graph = empty
repeat:
    for (each statement s in program):
        apply the rule for s to graph
until graph stops changing
```

- Every rule only **adds** edges — a **weak update**.

Andersen's Algorithm (3/3)

```
graph = empty
repeat:
    for (each statement s in program):
        apply the rule for s to graph
until graph stops changing
```

- Every rule only **adds** edges — a **weak update**.
- The graph is finite and grows monotonically, so it must stop.

Rule 1: Object Allocation (1/3)

Before



```
v = new B
```

After

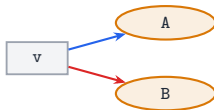
Rule 1: Object Allocation (2/3)

Before



```
v = new B
```

After



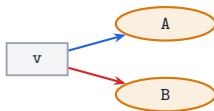
Rule 1: Object Allocation (3/3)

Before



```
v = new B
```

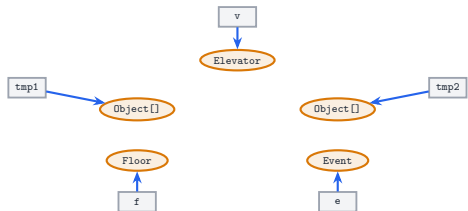
After



The old edge to A **stays**. A strong update would have removed it.

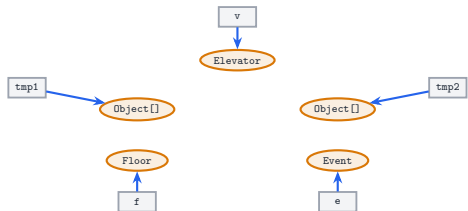
Rule 1 Applied: Five Allocations (1/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



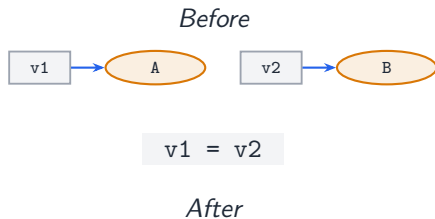
Rule 1 Applied: Five Allocations (2/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```

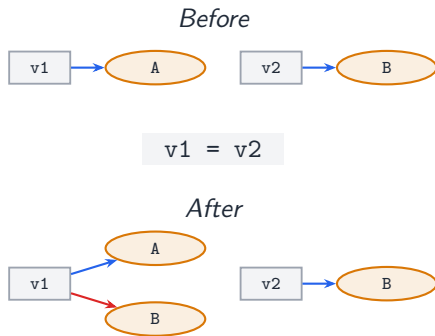


Two *separate* array nodes: the allocations are at **different sites**, even though the type is the same.

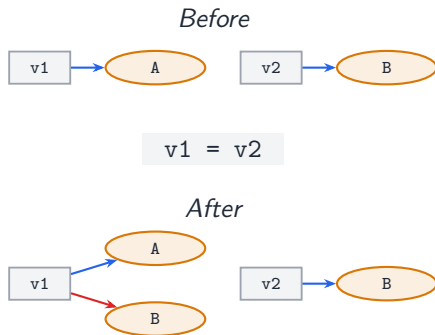
Rule 2: Object Copy (1/3)



Rule 2: Object Copy (2/3)

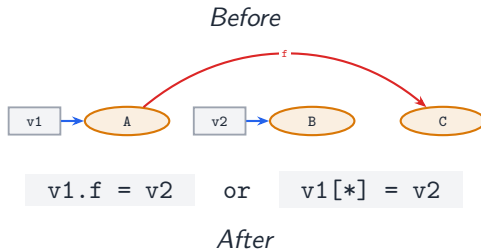


Rule 2: Object Copy (3/3)



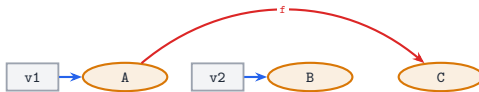
$v1$ gains every object $v2$ points to. In set terms: $\text{pts}(v1) \supseteq \text{pts}(v2)$.

Rule 3: Field Write (1/4)



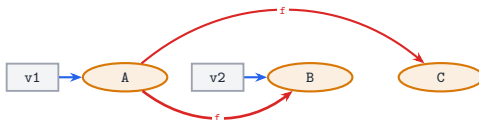
Rule 3: Field Write (2/4)

Before

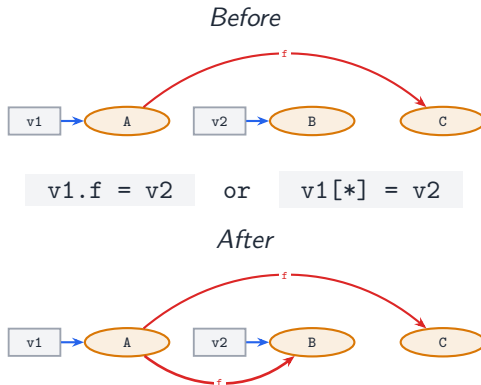


`v1.f = v2` or `v1[*] = v2`

After

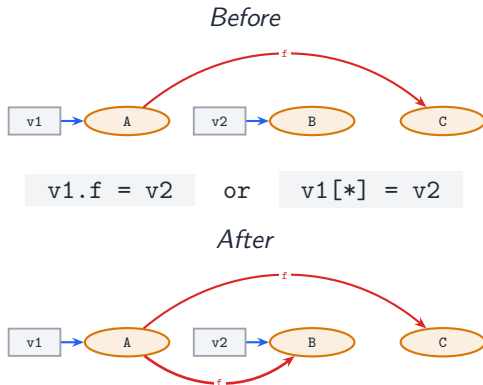


Rule 3: Field Write (3/4)



`v1`, `v2` same node? `v1.f = v1` is a **self-loop**.

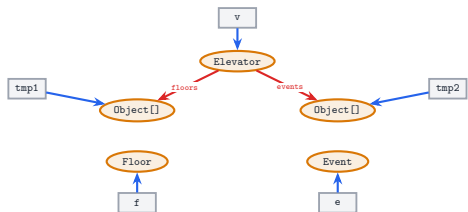
Rule 3: Field Write (4/4)



`v1`, `v2` same node? `v1.f = v1` is a **self-loop**.

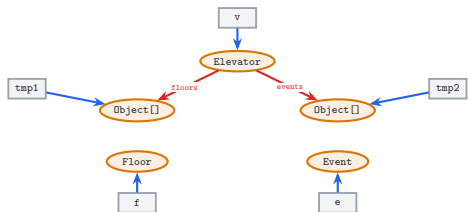
Rule 3 Applied: The Two Field Writes (1/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



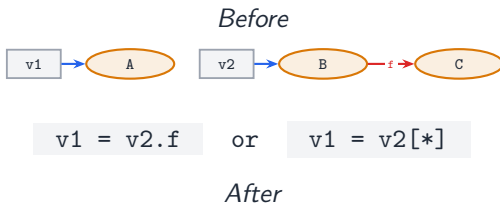
Rule 3 Applied: The Two Field Writes (2/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```

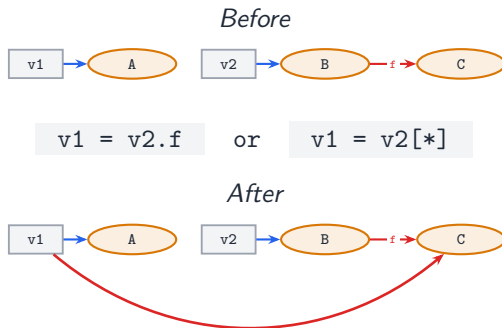


tmp3 and tmp4 have no nodes yet, so those two writes are **skipped** — for now.

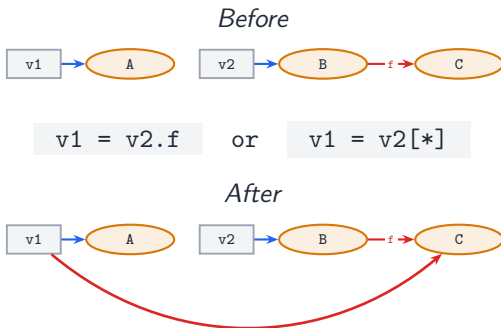
Rule 4: Field Read (1/3)



Rule 4: Field Read (2/3)



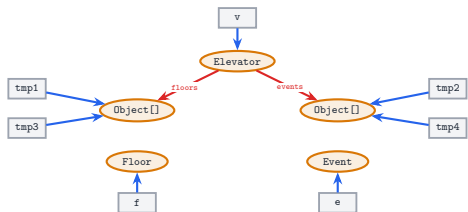
Rule 4: Field Read (3/3)



Follow every *f* edge out of every object *v2* reaches. Two hops, and the cost of that shows up later.

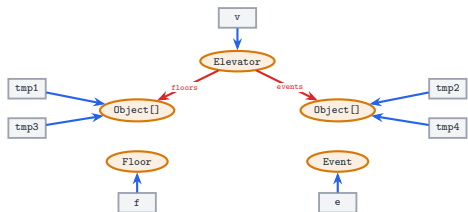
Rule 4 Applied: Reaching the Arrays (1/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



Rule 4 Applied: Reaching the Arrays (2/2)

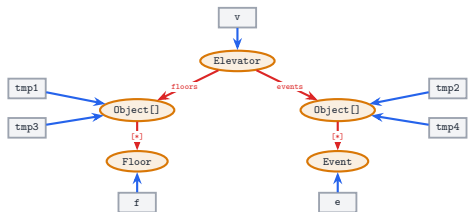
```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



`tmp3` lands on the *same* array node as `tmp1` — reached the long way, through the `Elevator`.

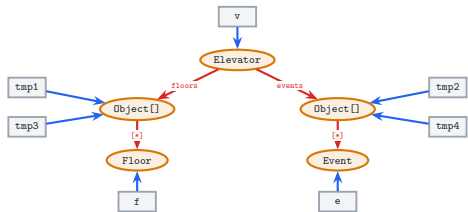
Closing the Fixed Point (1/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



Closing the Fixed Point (2/2)

```
v = new Elevator
tmp1 = new Object[]
v.floors = tmp1
tmp2 = new Object[]
v.events = tmp2
f = new Floor
tmp3 = v.floors
tmp3[*] = f
e = new Event
tmp4 = v.events
tmp4[*] = e
```



A third pass adds nothing. **Done** — eleven statements, two passes, eleven edges.

Andersen's Rules as Set Constraints (1/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint
-----------	------------

<code>v = new;</code>	
-----------------------	--

<code>v1 = v2</code>	
----------------------	--

<code>v1.f = v2</code>	
------------------------	--

<code>v1 = v2.f</code>	
------------------------	--

Andersen's Rules as Set Constraints (2/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint
-----------	------------

$v = new_i$	$i \in pts(v)$
-------------	----------------

$v1 = v2$	
-----------	--

$v1.f = v2$	
-------------	--

$v1 = v2.f$	
-------------	--

Rule 1

Andersen's Rules as Set Constraints (3/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint	
$v = \text{new}_i$	$i \in pts(v)$	Rule 1
$v1 = v2$	$pts(v2) \subseteq pts(v1)$	Rule 2
$v1.f = v2$		
$v1 = v2.f$		

Andersen's Rules as Set Constraints (4/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint	
$v = \text{new}_i$	$i \in pts(v)$	Rule 1
$v1 = v2$	$pts(v2) \subseteq pts(v1)$	Rule 2
$v1.f = v2$	for each $o \in pts(v1)$: $pts(v2) \subseteq pts(o.f)$	Rule 3
$v1 = v2.f$		

Andersen's Rules as Set Constraints (5/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint	
$v = \text{new}_i$	$i \in pts(v)$	Rule 1
$v1 = v2$	$pts(v2) \subseteq pts(v1)$	Rule 2
$v1.f = v2$	for each $o \in pts(v1)$: $pts(v2) \subseteq pts(o.f)$	Rule 3
$v1 = v2.f$	for each $o \in pts(v2)$: $pts(o.f) \subseteq pts(v1)$	Rule 4

Andersen's Rules as Set Constraints (6/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint	
$v = \text{new}_i$	$i \in pts(v)$	Rule 1
$v1 = v2$	$pts(v2) \subseteq pts(v1)$	Rule 2
$v1.f = v2$	for each $o \in pts(v1)$: $pts(v2) \subseteq pts(o.f)$	Rule 3
$v1 = v2.f$	for each $o \in pts(v2)$: $pts(o.f) \subseteq pts(v1)$	Rule 4

Arrays: read f as $[*]$. Only $\in$ and $\subseteq$ appear — nothing can **shrink** a set.

Andersen's Rules as Set Constraints (7/7)

Read the graph as sets. $pts(v)$ is where v 's blue edges go; $pts(o.f)$ is where oval o 's red f edges go.

Statement	Constraint	
$v = \text{new}_i$	$i \in pts(v)$	Rule 1
$v1 = v2$	$pts(v2) \subseteq pts(v1)$	Rule 2
$v1.f = v2$	for each $o \in pts(v1)$: $pts(v2) \subseteq pts(o.f)$	Rule 3
$v1 = v2.f$	for each $o \in pts(v2)$: $pts(o.f) \subseteq pts(v1)$	Rule 4

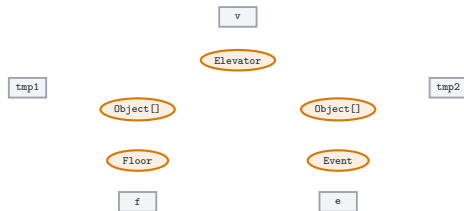
Arrays: read f as $[*]$. Only $\in$ and $\subseteq$ appear — nothing can **shrink** a set.

Andersen computes the **smallest** sets that satisfy every constraint at once.

The Elevator Fixed Point, as Sets (1/6)

Start: every set is empty ($\perp$).

Set	Value
<i>pts</i> (v)	$\emptyset$
<i>pts</i> (tmp1)	$\emptyset$
<i>pts</i> (tmp2)	$\emptyset$
<i>pts</i> (f)	$\emptyset$
<i>pts</i> (e)	$\emptyset$
<i>pts</i> (Elevator.floors)	$\emptyset$
<i>pts</i> (Elevator.events)	$\emptyset$
<i>pts</i> (tmp3)	$\emptyset$
<i>pts</i> (tmp4)	$\emptyset$
<i>pts</i> (arr1.[*])	$\emptyset$
<i>pts</i> (arr2.[*])	$\emptyset$

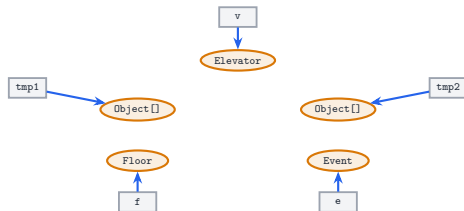


arr1, arr2: the two Object [] sites. Red marks a value this step changed.

The Elevator Fixed Point, as Sets (2/6)

After Rule 1: the five allocations.

Set	Value
<i>pts</i> (v)	{Elevator}
<i>pts</i> (tmp1)	{arr1}
<i>pts</i> (tmp2)	{arr2}
<i>pts</i> (f)	{Floor}
<i>pts</i> (e)	{Event}
<i>pts</i> (Elevator.floors)	∅
<i>pts</i> (Elevator.events)	∅
<i>pts</i> (tmp3)	∅
<i>pts</i> (tmp4)	∅
<i>pts</i> (arr1.[*])	∅
<i>pts</i> (arr2.[*])	∅



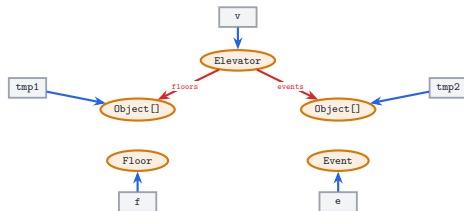
arr1, arr2: the two `Object[]` sites. Red marks a value this step changed.

The Elevator Fixed Point, as Sets (3/6)

After Rule 3: the two field writes.

Set	Value
<i>pts</i> (v)	{Elevator}
<i>pts</i> (tmp1)	{arr1}
<i>pts</i> (tmp2)	{arr2}
<i>pts</i> (f)	{Floor}
<i>pts</i> (e)	{Event}
<i>pts</i> (Elevator.floors)	{arr1}
<i>pts</i> (Elevator.events)	{arr2}
<i>pts</i> (tmp3)	∅
<i>pts</i> (tmp4)	∅
<i>pts</i> (arr1.[*])	∅
<i>pts</i> (arr2.[*])	∅

arr1, arr2: the two Object[] sites. **Red** marks a value this step changed.

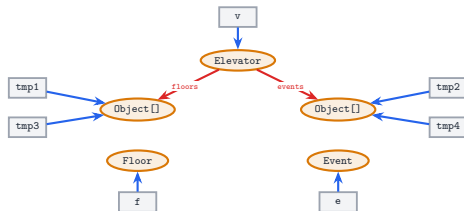


The Elevator Fixed Point, as Sets (4/6)

After Rule 4: the two field reads.

Set	Value
<i>pts</i> (v)	{Elevator}
<i>pts</i> (tmp1)	{arr1}
<i>pts</i> (tmp2)	{arr2}
<i>pts</i> (f)	{Floor}
<i>pts</i> (e)	{Event}
<i>pts</i> (Elevator.floors)	{arr1}
<i>pts</i> (Elevator.events)	{arr2}
<i>pts</i> (tmp3)	{arr1}
<i>pts</i> (tmp4)	{arr2}
<i>pts</i> (arr1.[*])	∅
<i>pts</i> (arr2.[*])	∅

arr1, arr2: the two Object[] sites. **Red** marks a value this step changed.

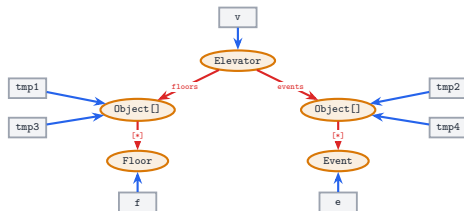


The Elevator Fixed Point, as Sets (5/6)

Pass 2, Rule 3: the two array writes. Nothing else changes.

Set	Value
<i>pts</i> (v)	{Elevator}
<i>pts</i> (tmp1)	{arr1}
<i>pts</i> (tmp2)	{arr2}
<i>pts</i> (f)	{Floor}
<i>pts</i> (e)	{Event}
<i>pts</i> (Elevator.floors)	{arr1}
<i>pts</i> (Elevator.events)	{arr2}
<i>pts</i> (tmp3)	{arr1}
<i>pts</i> (tmp4)	{arr2}
<i>pts</i> (arr1.[*])	{Floor}
<i>pts</i> (arr2.[*])	{Event}

arr1, arr2: the two Object[] sites. Red marks a value this step changed.

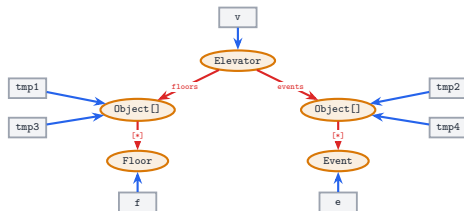


The Elevator Fixed Point, as Sets (6/6)

Fixed point: **11 elements, one per edge.**

Set	Value
$pts(v)$	{Elevator}
$pts(tmp1)$	{arr1}
$pts(tmp2)$	{arr2}
$pts(f)$	{Floor}
$pts(e)$	{Event}
$pts(Elevator.floors)$	{arr1}
$pts(Elevator.events)$	{arr2}
$pts(tmp3)$	{arr1}
$pts(tmp4)$	{arr2}
$pts(arr1.[*])$	{Floor}
$pts(arr2.[*])$	{Event}

arr1, arr2: the two Object[] sites. Red marks a value this step changed.



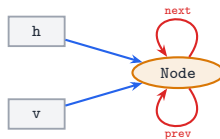
Quiz: A Doubly Linked List (1/4)

```
Node h = null;
while (more()) {
    Node v = new Node();
    v.next = h;
    if (h != null) h.prev = v;
    h = v;
}
```

Abstract it, then run Andersen.
What is the final graph?

Quiz: A Doubly Linked List (2/4)

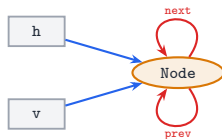
```
Node h = null;
while (more()) {
    Node v = new Node();
    v.next = h;
    if (h != null) h.prev = v;
    h = v;
}
```



Abstract it, then run Andersen.
What is the final graph?

Quiz: A Doubly Linked List (3/4)

```
Node h = null;
while (more()) {
    Node v = new Node();
    v.next = h;
    if (h != null) h.prev = v;
    h = v;
}
```

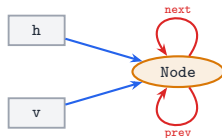


Abstract it, then run Andersen.
What is the final graph?

`h = null` adds no edge, so we drop it.

Quiz: A Doubly Linked List (4/4)

```
Node h = null;
while (more()) {
    Node v = new Node();
    v.next = h;
    if (h != null) h.prev = v;
    h = v;
}
```



Abstract it, then run Andersen.
What is the final graph?

`h = null` adds no edge, so we drop it.

One new site, so **one** Node oval — and both fields point back to it.

Four Dimensions of Classification (1/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

Four Dimensions of Classification (2/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

1. **Flow sensitivity** — does it respect statement order *within* a function?

Four Dimensions of Classification (3/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

1. **Flow sensitivity** — does it respect statement order *within* a function?
2. **Context sensitivity** — does it separate different callers of a function?

Four Dimensions of Classification (4/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

1. **Flow sensitivity** — does it respect statement order *within* a function?
2. **Context sensitivity** — does it separate different callers of a function?
3. **Heap abstraction** — how are objects grouped into ovals?

Four Dimensions of Classification (5/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

1. **Flow sensitivity** — does it respect statement order *within* a function?
2. **Context sensitivity** — does it separate different callers of a function?
3. **Heap abstraction** — how are objects grouped into ovals?
4. **Aggregate modelling** — how are arrays and fields modelled?

Four Dimensions of Classification (6/6)

Every pointer analysis can be placed on four axes. Each is a **precision** versus **cost** trade.

1. **Flow sensitivity** — does it respect statement order *within* a function?
2. **Context sensitivity** — does it separate different callers of a function?
3. **Heap abstraction** — how are objects grouped into ovals?
4. **Aggregate modelling** — how are arrays and fields modelled?

Naming a pointer analysis in the literature means giving its coordinates on these four axes.

Dimension 1: Flow Sensitivity (1/6)

	Flow-insensitive	Flow-sensitive
Program is		
Result is		
Updates are		
Cost		
Suits		

Dimension 1: Flow Sensitivity (2/6)

	Flow-insensitive	Flow-sensitive
Program is	an unordered set	a control-flow graph
Result is		
Updates are		
Cost		
Suits		

Dimension 1: Flow Sensitivity (3/6)

	Flow-insensitive	Flow-sensitive
Program is	an unordered set	a control-flow graph
Result is	one global graph	one graph per point
Updates are		
Cost		
Suits		

Dimension 1: Flow Sensitivity (4/6)

	Flow-insensitive	Flow-sensitive
Program is	an unordered set	a control-flow graph
Result is	one global graph	one graph per point
Updates are	weak — only add	strong — may kill
Cost		
Suits		

Dimension 1: Flow Sensitivity (5/6)

	Flow-insensitive	Flow-sensitive
Program is	an unordered set	a control-flow graph
Result is	one global graph	one graph per point
Updates are	weak — only add	strong — may kill
Cost	low	high
Suits	may-alias	must-alias

Dimension 1: Flow Sensitivity (6/6)

	Flow-insensitive	Flow-sensitive
Program is	an unordered set	a control-flow graph
Result is	one global graph	one graph per point
Updates are	weak — only add	strong — may kill
Cost	low	high
Suits	may-alias	must-alias

Ours is **flow-insensitive** — and it still proved `e` and `f` never alias.

Dimension 2: Context Sensitivity (1/6)

Control flow *between* functions — our elevator had only one.

Dimension 2: Context Sensitivity (2/6)

Control flow *between* functions — our elevator had only one.

- **Context-insensitive**: each function analysed *once*.

Dimension 2: Context Sensitivity (3/6)

Control flow *between* functions — our elevator had only one.

- **Context-insensitive**: each function analysed *once*.
- Cheap — but two callers get one blurred answer.

Dimension 2: Context Sensitivity (4/6)

Control flow *between* functions — our elevator had only one.

- **Context-insensitive**: each function analysed *once*.
- Cheap — but two callers get one blurred answer.
- **Context-sensitive**: once per *calling context*.

Dimension 2: Context Sensitivity (5/6)

Control flow *between* functions — our elevator had only one.

- **Context-insensitive**: each function analysed *once*.
- Cheap — but two callers get one blurred answer.
- **Context-sensitive**: once per *calling context*.
- Precise, expensive. A context: call site, object, type.

Dimension 2: Context Sensitivity (6/6)

Control flow *between* functions — our elevator had only one.

- **Context-insensitive**: each function analysed *once*.
- Cheap — but two callers get one blurred answer.
- **Context-sensitive**: once per *calling context*.
- Precise, expensive. A context: call site, object, type.

Ours is **context-insensitive**.